TALLER DE GO · 42 MÁLAGA · SESIÓN 2 · 12 DE NOVIEMBRE DE 2026

Bloque 1 · Almacenamiento con SQLite

En la sesión 1 el `Store` guardaba los productos y las reservas en mapas: al parar el programa se perdía todo. En este bloque el mismo `Store` guarda en un fichero SQLite. Las operaciones y los errores son los mismos; cambia dónde viven los datos.

Todo el trabajo está en `store.go`. Escribes ocho consultas SQL y seis métodos. Los tests de `store_test.go` te dicen cuándo está bien cada uno.

Lo que viene hecho

Antes de escribir, lee esta parte: es el código que ya está en `store.go` y que vas a copiar como modelo.

Las tablas

```
CREATE TABLE IF NOT EXISTS products (
    id    TEXT PRIMARY KEY,
    name  TEXT NOT NULL,
    stock INTEGER NOT NULL
);
CREATE TABLE IF NOT EXISTS reservations (
    id         INTEGER PRIMARY KEY,
    product_id TEXT NOT NULL REFERENCES products(id),
    quantity   INTEGER NOT NULL,
    status     INTEGER NOT NULL
);
```

Es la constante `schema`. Se ejecuta cada vez que se abre el fichero; con `IF NOT EXISTS`, si las tablas ya están no cambia nada.

- El `id` de un producto lo pones tú: `"p1"`. El de una reserva lo elige la base de datos: 1, 2, 3.
- `status` guarda un número: `Held` es 0, `Confirmed` es 1 y `Canceled` es 2.
- Los nombres de tabla y de columna de tus consultas salen de aquí.

Conectarse a la base de datos

```
import (
    "database/sql"

    _ "modernc.org/sqlite" // el driver de SQLite
)

func OpenStore(path string) (*Store, error) {
    db, err := sql.Open("sqlite", "file:"+path+"?...")
    if err != nil {
        return nil, err
    }
    if _, err := db.Exec(schema); err != nil {
        db.Close()
        return nil, err
    }
    return &Store{db: db}, nil
}
```

- `database/sql` es de la biblioteca estándar y sirve para cualquier base de datos. Lo único propio de SQLite es el driver: se importa con `_` y se pide por su nombre, `"sqlite"`.
- `sql.Open` recibe el nombre del driver y dónde está la base de datos. Aquí es un fichero; si no existe, se crea.
- Devuelve un `*sql.DB`. Se abre una sola vez y se guarda en el `Store`: todos los métodos lo usan como `s.db`.
- `db.Exec(schema)` crea las tablas.

Conectarse a otra base de datos

Con PostgreSQL o MySQL cambian dos cosas: el driver que se importa y los dos argumentos de `sql.Open`. En vez de un fichero, se indica el servidor, el usuario y el nombre de la base de datos.

PostgreSQL:

```
import (
    "database/sql"

    _ "github.com/jackc/pgx/v5/stdlib" // el driver de PostgreSQL
)

db, err := sql.Open("pgx", "postgres://usuario:clave@localhost:5432/taquilla")
```

MySQL:

```
import (
    "database/sql"

    _ "github.com/go-sql-driver/mysql" // el driver de MySQL
)

db, err := sql.Open("mysql", "usuario:clave@tcp(localhost:3306)/taquilla")
```

- Lo que viene después es igual: el mismo `*sql.DB` y las mismas llamadas para lanzar consultas.

- El SQL sí cambia un poco de una base de datos a otra. Por ejemplo, PostgreSQL marca los valores con `$1`, `$2` en vez de `?`.
- En el taller solo usamos SQLite: no hay que instalar ni arrancar ningún servidor.

Lanzar una consulta

Hay tres llamadas, y con ellas se hace todo el bloque:

LLAMADA	PARA QUÉ	QUÉ DEVUELVE
`s.db.Exec(consulta, args...)`	`INSERT` y `UPDATE`: no devuelven filas	Un `Result` y un error
`s.db.QueryRow(consulta, args...)`	Un `SELECT` de una sola fila	Una fila, que se lee con `Scan`
`s.db.Query(consulta, args...)`	Un `SELECT` de varias filas	Las filas, que se recorren con un bucle

Los valores nunca se pegan dentro del texto de la consulta. En la consulta se pone un `?` por cada valor, y los valores se pasan como argumentos, en el mismo orden:

```
const sqlSelectStock = `SELECT stock FROM products WHERE id = ?`

s.db.QueryRow(sqlSelectStock, productID)   // productID ocupa el ?
```

Las nueve consultas están en un bloque `const` al principio de `store.go`. Una está escrita, `sqlSelectStock`. Las otras ocho dicen `TODO` y son tuyas: el comentario de cada una dice qué tiene que hacer.

Leer una fila: `Stock`

```
func (s *Store) Stock(productID string) (int, error) {
    var stock int
    err := s.db.QueryRow(sqlSelectStock, productID).Scan(&stock)
    if errors.Is(err, sql.ErrNoRows) {
        return 0, ErrProductNotFound
    }
    if err != nil {
        return 0, err
    }
    return stock, nil
}
```

- `Scan` copia las columnas de la fila en tus variables. Recibe punteros (`&stock`) porque escribe en ellas. Una variable por columna, en el orden del `SELECT`.
- Si la consulta no encuentra ninguna fila, el error es `sql.ErrNoRows`. Aquí quiere decir que el producto no existe, así que se convierte en `ErrProductNotFound`.
- Cualquier otro error se devuelve tal cual.

Leer muchas filas: `Products`

```
func (s *Store) Products() ([]*Product, error) {
    rows, err := s.db.Query(sqlSelectProducts)
    if err != nil {
        return nil, err
    }
    defer rows.Close()
    ps := make([]*Product, 0)
    for rows.Next() {
        p := &Product{}
        if err := rows.Scan(&p.ID, &p.Name, &p.Stock); err != nil {
            return nil, err
        }
        ps = append(ps, p)
    }
    return ps, rows.Err()
}
```

- `Query` devuelve `rows`. Se cierran siempre: `defer rows.Close()`.
- `rows.Next()` avanza una fila y devuelve `false` cuando no quedan más. Dentro del bucle, `rows.Scan` lee la fila actual.
- `rows.Err()` dice si el recorrido acabó bien o se cortó a medias.
- El método está escrito, pero su consulta, `sqlSelectProducts`, es tuya (paso 2).

Cambiar un estado: `Confirm`

```
func (s *Store) Confirm(reservationID string) error {
    r, err := s.Reservation(reservationID)   // 1. cargar
    if err != nil {
        return err
    }
    if err := r.confirm(); err != nil {      // 2. decidir
        return err
    }
    _, err = s.db.Exec(sqlUpdateStatus, r.Status, reservationID)   // 3. guardar
    return err
}
```

- `r.confirm()` es el método de la sesión 1: sabe qué cambios de estado son legales. El `Store` no repite esa regla.
- Si el cambio es ilegal, se sale antes del `Exec` y no se escribe nada.
- El método está escrito, pero su consulta, `sqlUpdateStatus`, es tuya (paso 4).

También viene hecho `scanReservation`: recibe una fila de `reservations`, la convierte en una `Reservation` y, si no hay fila, devuelve `ErrReservationNotFound`.

Paso a paso

Cada paso empieza por el test que lo comprueba. Lánzalo antes de escribir, para verlo fallar, y otra vez al acabar. No pases al siguiente paso hasta que pase.

Paso 1 · `AddProduct` (lo hacemos juntos)

```
go test -run TestAddProduct -v
```

- [ ] `sqlInsertProduct`: inserta un producto con sus tres columnas.
- [ ] `AddProduct`: guarda el producto `p` con esa consulta.

Modelo: la última línea de `Confirm`, que también escribe y solo mira el error.

Paso 2 · `Product` y la consulta de `Products`

```
go test -run TestProduct -v
```

- [ ] `sqlSelectProduct`: un producto por su `id`.
- [ ] `sqlSelectProducts`: todos los productos, ordenados por `id`. El método `Products` ya la usa: mira en su `Scan` qué columnas espera y en qué orden.
- [ ] `Product`: lee un producto por su `id`. Si no existe, devuelve `ErrProductNotFound`.

Modelo: `Stock`, con tres columnas en vez de una.

Paso 3 · `Reserve` (el paso largo)

```
go test -run TestReserve -v
```

- [ ] `sqlUpdateStock`: suma una cantidad al stock de un producto. Con una cantidad negativa resta: la misma consulta sirve para reservar y para devolver.
- [ ] `sqlInsertReservation`: inserta una reserva. El `id` no se pasa: lo elige la base de datos.
- [ ] `Reserve`: descuenta el stock, guarda la reserva en estado `Held` y la devuelve. Antes comprueba tres cosas, en este orden:
  1. La cantidad es cero o negativa: `ErrInvalidQuantity`.
  2. El producto no existe: `ErrProductNotFound`.
  3. No hay stock suficiente: `ErrOutOfStock`.

Modelos: `Stock` para la lectura y la última línea de `Confirm` para las dos escrituras.

El `id` que eligió la base de datos lo da el `Result` del `INSERT`: `res.LastInsertId()`. Es un `int64` y la reserva lo guarda como texto: `strconv.FormatInt(id, 10)`.

Paso 4 · `Reservation` y la consulta de `Confirm`

```
go test -run 'TestReservation_' -v
go test -run TestConfirm -v
```

- [ ] `sqlSelectReservation`: una reserva por su `id`. `scanReservation` ya está escrita: mira qué columnas lee y en qué orden.
- [ ] `sqlUpdateStatus`: cambia el estado de una reserva. `Confirm` ya la usa: mira en qué orden le pasa los valores.
- [ ] `Reservation`: lee una reserva por su `id`. El `Scan` y el error de cuando no hay fila los pone `scanReservation`.

Paso 5 · `Cancel`

```
go test -run TestCancel -v
```

- [ ] `Cancel`: cancela la reserva y devuelve sus unidades al stock del producto.

Modelo: `Confirm`. Si la reserva se puede cancelar lo decide `r.cancel()`; cuando devuelve un error no se escribe nada, y por eso cancelar dos veces no devuelve el stock dos veces. No hace falta ninguna consulta nueva.

Paso 6 · `Reservations`

```
go test -run TestReservations -v
```

- [ ] `sqlSelectReservations`: todas las reservas de un producto, la más antigua primero.
- [ ] `Reservations`: devuelve esas reservas.

Modelo: `Products`. El `id` de cada reserva se convierte a texto, como en `scanReservation`. Si el producto no tiene reservas o no existe, el resultado es un slice vacío, no `nil`.

Comprobar que los datos siguen ahí

```
go test -run TestStore_survivesReopen -v
```

Aquí no se escribe nada: si los seis pasos están bien, pasa. El test cierra el `Store`, abre otra vez el mismo fichero y comprueba que el stock y la reserva siguen como estaban.

Comprobar el bloque entero

```
go test -run
'TestAddProduct|TestStock|TestProduct|TestReserv|TestConfirm|TestCancel|TestStore' -v
```

Has terminado cuando ese comando pasa y en `store.go` no queda ningún `TODO`.

Los tests que empiezan por `TestAPI` y `TestConcurrent` son de los bloques 2 y 3: por eso `go test ./...` sigue fallando.

Si algo falla

EL TEST DICE	QUÉ PASA
`near "TODO": syntax error`	El método usa una consulta que sigue en `TODO`. Mira qué consulta usa y escríbela.
`TODO: Reserve` (o el nombre de otro método)	El método sigue sin escribir.
`no such column` o `no such table`	Un nombre de la consulta no coincide con el de las tablas.
`expected 3 destination arguments in Scan, not 1`	La consulta devuelve más o menos columnas de las que lee `Scan`.